Exp:6 - Temperature Analysis using MapReduce

1. Objective

To implement a Hadoop MapReduce program that finds the maximum temperature recorded for each year.

2. Environment

- WSL Ubuntu
- Hadoop 3.3.6
- Java 8 runtime
- Eclipse / terminal (optional)
- HDFS for input and output

3. Dataset

Create a file named temperature.csv:

```
Year,Month,Day,Temperature
2024,01,01,23
2024,01,02,27
2024,01,03,25
2023,01,01,22
2023,01,02,30
2023,01,03,21
```

```
Start-dfs.sh

Start-yarn.sh
```

4. Create the Project

```
cd ~/eclipse-workspace
mkdir -p TemperatureAnalysis
cd TemperatureAnalysis
```

5. TemperatureMapper.java

```
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;

public class TemperatureMapper
        extends Mapper<LongWritable, Text, Text, IntWritable> {

    private Text year = new Text();
    private IntWritable temperature = new IntWritable();

    @Override
    protected void map(LongWritable key, Text value, Context context)
```

```
            throws IOException, InterruptedException {

        String line = value.toString().trim();

        // Skip the header
        if (line.startsWith("Year,")) {
            return;
        }

        String[] fields = line.split(",");

        if (fields.length >= 4) {
            year.set(fields[0]);
            temperature.set(Integer.parseInt(fields[3]));
            context.write(year, temperature);
        }
    }
}
```

The Mapper extracts year and temperature and emits them as key-value pairs, matching the supplied solution.

6. TemperatureReducer.java

```
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;

public class TemperatureReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values,
                          Context context)
            throws IOException, InterruptedException {

        int maxTemperature = Integer.MIN_VALUE;

        for (IntWritable value : values) {
            maxTemperature = Math.max(maxTemperature, value.get());
        }

        context.write(key, new IntWritable(maxTemperature));
    }
}
```

The Reducer finds the maximum temperature for each year. □filecite□turn5file0□L63-L84□

7. TemperatureAnalysis.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```
public class TemperatureAnalysis {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.err.println(
                "Usage: TemperatureAnalysis <input path> <output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Temperature Analysis");

        job.setJarByClass(TemperatureAnalysis.class);
        job.setMapperClass(TemperatureMapper.class);
        job.setReducerClass(TemperatureReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Driver configures the MapReduce job, input path, output path, Mapper and Reducer.

8. Create the Dataset

```
cd ~/eclipse-workspace/TemperatureAnalysis
nano temperature.csv
```

Paste the dataset from Section 3, save with Ctrl+O, press Enter, then exit with Ctrl+X.

9. Compile

```
export HADOOP_CLASSPATH=$(hadoop classpath)

javac -classpath "$HADOOP_CLASSPATH" -d . TemperatureMapper.java TemperatureReducer.java
TemperatureAnalysis.java

find . -name "*.class"
```

10. Create the JAR

```
rm -f TemperatureAnalysis.jar
jar -cvf TemperatureAnalysis.jar *.class
jar -tf TemperatureAnalysis.jar
```

11. Start Hadoop

```
start-dfs.sh
start-yarn.sh
jps
```

12. Upload Dataset to HDFS

```
hdfs dfs -mkdir -p /input
hdfs dfs -put -f temperature.csv /input/
hdfs dfs -ls /input
hdfs dfs -cat /input/temperature.csv
```

13. Run MapReduce

The supplied solution uses the TemperatureAnalysis JAR with /input/temperature.csv and /output/temperature_analysis. □filecite□turn5file0□L135-L136□

```
hdfs dfs -rm -r /output/temperature_analysis

hadoop jar TemperatureAnalysis.jar TemperatureAnalysis /input/temperature.csv
/output/temperature_analysis
```

14. View Output

```
hdfs dfs -ls /output/temperature_analysis
hdfs dfs -cat /output/temperature_analysis/part-r-00000
```

15. Expected Output

```
2023    30
2024    27
```

These are the maximum temperatures stated in the supplied solution. □filecite□turn5file0□L139-L142□

16. Complete Command Sequence

```
cd ~/eclipse-workspace
mkdir -p TemperatureAnalysis
cd TemperatureAnalysis

export HADOOP_CLASSPATH=$(hadoop classpath)

javac -classpath "$HADOOP_CLASSPATH" -d . TemperatureMapper.java TemperatureReducer.java
TemperatureAnalysis.java

rm -f TemperatureAnalysis.jar
jar -cvf TemperatureAnalysis.jar *.class

start-dfs.sh
start-yarn.sh

hdfs dfs -mkdir -p /input
hdfs dfs -put -f temperature.csv /input/

hdfs dfs -rm -r /output/temperature_analysis

hadoop jar TemperatureAnalysis.jar TemperatureAnalysis /input/temperature.csv
/output/temperature_analysis

hdfs dfs -cat /output/temperature_analysis/part-r-00000
```

17. Troubleshooting

- **Connection refused:** Run start-dfs.sh and check Hadoop processes using jps.
- **Output directory already exists:** Run hdfs dfs -rm -r /output/temperature_analysis.
- **ClassNotFoundException:** Check the class name and verify the JAR using jar -tf TemperatureAnalysis.jar.
- **FileInputFormat/FileOutputFormat not found:** Ensure the Driver imports org.apache.hadoop.mapreduce.lib.input.FileInputFormat and org.apache.hadoop.mapreduce.lib.output.FileOutputFormat.

18. Result

The Temperature Analysis MapReduce program was implemented and executed using WSL Ubuntu with Hadoop 3.3.6. It groups temperature readings by year and finds the maximum temperature for each year.